

Qué son

Las **listas** en Python son **colecciones ordenadas y mutables** que permiten almacenar múltiples elementos (números, textos, booleanos, etc.) en una sola variable.

👉 Se definen con corchetes `[]`.

```
frutas = ["manzana", "pera", "naranja"]
```

Son **esenciales** en proyectos modulares porque permiten modificar, ordenar y buscar elementos de forma eficiente durante la ejecución del programa

◆ Tipos de listas

Tipo de lista	Descripción	Ejemplo
Simple s	Contienen elementos del mismo tipo (homogéneos).	<code>[1, 2, 3, 4, 5]</code>
De diferentes tipos	Mezclan varios tipos de datos (enteros, cadenas, booleanos...).	<code>[23, "texto", True, 3.14]</code>
Anidadas o complejas	Contienen otras listas o estructuras dentro de sí.	<code>usuarios = [{"Juan", 30}, {"Ana", 25}]</code>

Las listas simples son más rápidas; las anidadas organizan mejor datos jerárquicos, y las mixtas son más flexibles.

Propiedades clave

- **Ordenadas:** mantienen el orden de inserción de los elementos.
- **Mutables:** pueden modificarse (añadir, eliminar o cambiar valores).
- **Admiten duplicados:** puedes tener varios elementos iguales.
- **Indexadas:** se accede a sus elementos por su posición.

Acceso y modificación de elementos

Operación	Descripción	Ejemplo
Acceso por índice	Accede al elemento en una posición concreta (empieza en 0).	<code>mi_lista[1]</code>
Modificación directa	Permite cambiar un elemento.	<code>mi_lista[2] = "nuevo"</code>
Slicing (sublistas)	Extrae o modifica partes de la lista.	<code>mi_lista[1:3], mi_lista[:2]</code>
Índices negativos	Permiten acceder desde el final.	<code>mi_lista[-1]</code>

 Ejemplo:

```
colores = ["rojo", "verde", "azul"]
colores[1] = "amarillo" # cambia 'verde' por 'amarillo'
print(colores)
# ['rojo', 'amarillo', 'azul']
```

Métodos comunes de manipulación

Método	Descripción	Ejemplo	Ejemplo de resultado
<code>append(x)</code>	Agrega un elemento al final de la lista. Si x es una lista, se añade como sublista → lista anidada	<code>mi_lista.append(8) mi_lista.append([1,2,3])</code>	<code>[8, [1,2,3]]</code>
<code>extend(x)</code>	Agrega varios elementos al final de la lista. Si x es una lista, agrega cada elemento individualmente → lista plana/mixta	<code>mi_lista.extend([8,9,10]) mi_lista.extend([1,2,3])</code>	<code>[8,9,10,1,2,3]</code>
<code>insert(i, x)</code>	Inserta un elemento en una posición concreta.	<code>mi_lista.insert(2, "dato")</code>	<code>[8,9,"dato",10]</code>
<code>remove(x)</code>	Elimina el primer valor igual a x.	<code>mi_lista.remove("dato")</code>	<code>[8,9,10]</code>

pop(i)	Elimina el elemento en la posición i y lo devuelve. Sin argumento, elimina el último.	<code>eliminado = mi_lista.pop(1)</code>	<code>eliminado=9</code> y <code>lista=[8,10]</code>
clear()	Vacía toda la lista.	<code>mi_lista.clear()</code>	<code>[]</code>
index(x)	Devuelve la posición de un valor.	<code>mi_lista.index(3)</code>	<code>2</code>
count(x)	Cuenta cuántas veces aparece un valor.	<code>mi_lista.count("rojo")</code>	<code>1</code>
sort()	Ordena la lista (alfabéticamente o numéricamente).	<code>mi_lista.sort()</code>	<code>[1,2,3,8,10]</code>
reverse()	Invierte el orden de los elementos.	<code>mi_lista.reverse()</code>	<code>[10,8,3,2,1]</code>
len()	Devuelve el número de elementos.	<code>len(mi_lista)</code>	<code>5</code>
in	Comprueba si un valor existe.	<code>"rojo" in colores</code>	<code>True</code>

Estos métodos permiten modificar listas sin crear nuevas estructuras, ahorrando memoria y tiempo

Ordenación y búsqueda

Técnica	Descripción	Ejemplo
<code>sort()</code>	Ordena la lista directamente.	<code>numeros.sort();</code> ✓ Modifica la lista original ✓ No devuelve nada (devuelve <code>None</code>) ✓ Ordena la lista <i>en el sitio</i> (in-place)
<code>sorted(lista)</code>	Crea una nueva lista ordenada, no modifica la original.	<code>sorted(numeros)</code> ✓ NO modifica la lista original ✓ Devuelve una nueva lista ordenada ✓ Sirve para mantener intacta la lista original
Ordenación con <code>key</code>	Ordena por un atributo o función.	<code>palabras = ["sol", "estrella", "luna", "planeta"]</code> <code>palabras.sort(key=len)</code> <code>print(palabras)</code> <code>['sol', 'luna', 'planeta', 'estrella']</code> Otro ejemplo: <code>nombres = ["ana", "Luis", "carlos", "Bea"]</code> <code>nombres.sort(key=str.lower) #ignora mayúsculas</code> <code>print(nombres)</code> <code>['ana', 'Bea', 'carlos', 'Luis']</code>

Búsqueda secuencial	Recorre la lista elemento a elemento.	<code>if "Ana" in lista:</code>
Búsqueda binaria	Requiere lista ordenada, busca más rápido.	(con módulo <code>bisect</code>)

💡 También puedes usar `min()`, `max()` o `sum()` para obtener valores específicos

IMPORTANTE sobre ordenar listas

- `sort()` solo funciona con listas cuyos elementos se pueden comparar entre sí, por ejemplo:
 - números con números
 - cadenas con cadenas

❌ **No funciona con listas mixtas** (porque no puede comparar `"hola"` con `10`)

❌ **No funciona con listas anidadas** con `sort()` (porque no sabe cómo comparar sublistas entre sí), lo único que podemos hacer en este caso es **ordenar las sublistas si no son mixtas**. Cuando Python ordena una lista con `.sort()` o `sorted()`, compara los elementos de la lista **entre sí** y los coloca en el orden adecuado según su valor. Si los elementos son números, los ordena de menor a mayor; si son cadenas, las ordena alfabéticamente. Cuando se trata de una lista anidada (listas dentro de otra lista), Python **no ordena el contenido interno de cada sublista**, sino que compara cada sublista como un bloque y decide su posición comparando primero su *primer elemento*; si estos son iguales, compara el segundo, y así sucesivamente. Por eso, para que una lista anidada se pueda ordenar, **todas las sublistas deben contener elementos del mismo tipo**, ya que Python necesita poder compararlas entre sí. En resumen, Python ordena moviendo los elementos completos de las sublistas, manteniendo intacto su contenido interno.

Tipo de lista	¿Se puede ordenar con <code>.sort()</code> ?	Explicación
Simple (solo números)	✓ Sí	Todos son comparables
Solo strings	✓ Sí	Se ordenan alfabéticamente

Anidadas	✓ Sí, si las sublistas son comparables	Python compara listas entre sí. Todas las sublistas deben ser del mismo tipo, números o strings.
Mixtas	✗ No	No se puede comparar "hola" con 10

Búsqueda binaria:

La búsqueda binaria sirve para **encontrar rápidamente un elemento en una lista ordenada**.

En lugar de revisar los elementos uno por uno (como en la búsqueda secuencial), la búsqueda binaria **divide la lista a la mitad en cada paso**, reduciendo el número de comparaciones.

Funciona **solo si la lista ya está ordenada** (por ejemplo, con `.sort()` o `sorted()`).

Cómo funciona:

Tienes una lista **ordenada**. Ejemplo:

```
numeros = [1, 3, 5, 7, 9, 11, 13]
```

1. Buscas un valor, por ejemplo 7.
 - Empieza en el **centro** de la lista.
 - Si el número buscado es menor, descarta la mitad derecha.
 - Si es mayor, descarta la mitad izquierda.
 - Repite el proceso hasta encontrarlo.
2. Repite el proceso con la mitad correspondiente hasta encontrar el valor o agotar la lista.

Iteración y filtrado

Las listas pueden recorrerse con bucles o filtrarse con comprensión de listas.

♦ Iterar (recorrer) con **for**

```
for elemento in mi_lista:  
    print(elemento)
```

El bucle **for** recorre (**itera**) la lista **mi_lista** elemento por elemento. En cada vuelta del bucle, la variable **elemento** **toma el valor** del siguiente elemento de la lista. Ejemplo:

```
mi_lista = ["rojo", "verde", "azul"]  
for elemento in mi_lista:  
    print(elemento)  
rojo  
verde  
azul
```

♦ Filtrar con comprensión de listas

```
pares = [n for n in range(10) if n % 2 == 0] #[nueva_variable for variable in secuencia if condición]  
print(pares) # [0, 2, 4, 6, 8]
```

Qué hace

① `range(10)`

→ genera los números del 0 al 9 (no incluye el 10).

 `[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`

② `for n in range(10)`

→ recorre cada número del rango.

③ `if n % 2 == 0`

→ solo selecciona los que son **pares** (% calcula el resto de la división entre 2).

- Si el resto es 0 → el número es par.

④ `[n for n in range(10) if n % 2 == 0]`

→ crea una lista con todos los **n** que cumplen la condición.

Resultado final

[0, 2, 4, 6, 8]

La primera **n** en esta expresión:

```
pares = [n for n in range(10) if n % 2 == 0]
```

es lo que se va a guardar en la lista.

◆ Funciones útiles

Funciones útiles

Función	Descripción	Ejemplo	Devuelve
map()	Aplica una función a todos los elementos.	<pre>def doble(x): return x * 2 numeros = [1, 2, 3, 4] resultado = list(map(doble, numeros)) #map() aplica la función doble() a cada elemento de la lista numeros.map() no devuelve directamente una lista, sino un objeto iterable (una especie de "receta" que genera los valores uno a uno). Por eso se usa list() para convertir ese resultado en una lista real que se pueda imprimir o usar.</pre>	[2, 4, 6, 8]
filter()	Filtra elementos según una condición.	<pre>def mayor_que_cinco(x): return x > 5 #Esa función recibe un número (x) y devuelve True o False según si el número es mayor que 5. numeros = [2, 7, 4, 9, 3, 8] resultado = list(filter(mayor_que_cinco, numeros)) #filter() siempre trabaja con valores booleanos, en nuestro ejemplo aplica la función mayor_que_cinco a cada elemento de la lista numeros. Solo mantiene los valores que devuelven True.</pre>	[7, 9, 8]

Función	Descripción
map()	Aplica una función a todos los elementos.
filter()	Filtra elementos según una condición.

Estas funciones son ideales para procesar grandes volúmenes de datos sin usar bucles manuales



Resumen general

Propiedad / Operación	Ejemplo de uso
Crear lista	<code>lista = [1, 2, 3]</code>
Añadir elemento	<code>lista.append(4)</code>
Insertar elemento	<code>lista.insert(1, "dato")</code>
Eliminar por valor	<code>lista.remove("dato")</code>
Eliminar por índice	<code>lista.pop(2)</code>
Acceder por índice	<code>lista[0]</code>
Sublista (slicing)	<code>lista[1:3]</code>
Buscar elemento	<code>"rojo" in lista</code>
Contar elementos	<code>len(lista)</code>
Ordenar	<code>lista.sort()</code>
Invertir	<code>lista.reverse()</code>
Filtrar	<code>[x for x in lista if x > 5]</code>

Conclusión

Las **listas en Python** son una herramienta **versátil, flexible y potente** para almacenar y manipular datos.

Gracias a su **mutabilidad**, pueden crecer, cambiar y adaptarse durante la ejecución del programa.

Dominar sus **métodos** (como `append`, `insert`, `remove`, `sort`, `index`, `count`, etc.) es clave para desarrollar código **eficiente, limpio y escalable**.

EJERCICIOS CON LISTAS

EJERCICIO 1 — Crear y mostrar una lista de cada tipo

Objetivo: Crear una lista simple, otra mixta y otra anidada

EJERCICIO 2 — Recorrer listas

Objetivo: Imprimir cada elemento de una lista simple, una lista mixta y una anidada con un `for`.

```
frutas = ["manzana", "pera", "plátano", "naranja"]
for fruta in frutas:
    print("Fruta:", fruta)
```

EJERCICIO 3 — Añadir y eliminar elementos

Objetivo: Usar `append()` y `remove()`.

```
colores = ["rojo", "verde", "azul"]
colores.append("amarillo")      # Añadir un color
print("Después de añadir:", colores)

colores.remove("verde")         # Eliminar un color
print("Después de eliminar:", colores)
```

EJERCICIO 4 — Ordenar listas

Objetivo: Ordenar una lista de números. `.sort()` y `.reverse()`.

```
numeros = [8, 3, 10, 1, 6]
numeros.sort()                  # Orden ascendente
print("Lista ordenada:", numeros)

numeros.reverse()               # Orden descendente
print("Lista en orden inverso:", numeros)
```

EJERCICIO 5 – Buscar un elemento

Objetivo: Comprobar si un valor está en la lista. `in`

```
animales = ["gato", "perro", "loro", "pez"]
buscado = input("Escribe un animal: ")

if buscado in animales:
    print("✅", buscado, "está en la lista.")
else:
    print("❌", buscado, "no está en la lista.")
```

EJERCICIO 6

Usando las funciones `map()` y `filter()`, escribe un programa en Python que:

1. A partir de una lista de números, genere **otra lista con el doble** de cada elemento.
2. Dada una segunda lista, obtenga **una nueva lista que contenga solo los números mayores que 5**.

Define las funciones necesarias y muestra las listas originales y las resultantes.

EJERCICIO 7

Escribe un programa en Python que realice lo siguiente:

1 . Lista simple:

Solicita al usuario que ingrese 5 números enteros y crea una lista que los contenga. Luego muéstrala por pantalla.

2 . Lista mixta:

Pide al usuario su nombre, su edad y su ciudad, y crea una lista que combine estos datos (distintos tipos). Muestra la lista resultante.

3 . Lista anidada:

Solicita al usuario que ingrese el nombre y la nota de tres estudiantes.
Con esta información, crea una lista anidada donde cada elemento sea a su vez una lista con la forma:
["nombre_del_estudiante", nota]
Muestra la lista anidada completa.

Añade código a este programa para pedir los datos a través de un bucle.

EJERCICIO 8.

8.A Gestión de árboles: decidir el abono según altura y edad. Listas simples paralelas.

Objetivo:

Pedir datos de varios árboles (altura y edad), guardarlos en listas, y decidir **qué abono necesitan** según sus características.

♦ Criterios de ejemplo:

- Si **mide más de 10 metros** → abono tipo **A**
- Si **tiene más de 20 años** → abono tipo **B**
- Si cumple las dos → abono **A + B**
- Si no cumple nada → **sin abono especial**

8.B Gestión de árboles: decidir el abono según altura y edad. Listas anidadas.